

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Training LLMs to search external knowledge is a central route toward agents that can answer complex, real-world questions beyond their static pretraining data. Existing approaches leave two contribution-critical gaps: prompting-only systems do not train the search policy, while RL-based search agents often hit performance ceilings or collapse under sparse, long-horizon feedback. We address these gaps with **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an RL algorithm that makes robust agent training explicit through two components: sequence-level policy optimization aligned with trajectory-level rewards, and dynamic outcome filtering that preserves informative rollout groups. Using only RL, DSPO trains a model to interleave multi-turn search and reasoning without supervised demonstrations. Across multiple QA benchmarks, our 7B model improves over a comparable prior 7B system by **34.1%**; on complex multihop QA such as HotpotQA, it also surpasses the prior 14B model by nearly **9% relative**, while maintaining stable training dynamics.

1 INTRODUCTION

LLMs (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) now perform strongly on specialized tasks such as math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024), but their factual knowledge remains bounded by pretraining. A search-capable LLM addresses this limitation by becoming an agent that can query external knowledge sources when its internal knowledge is insufficient (Jin et al., 2025b). This setting is a concrete and demanding form of tool use (Schick et al., 2023): the model must decide when to search, how to formulate queries, how to incorporate retrieved evidence, and when to answer. The first contribution need therefore is algorithmic rather than merely architectural: agentic search requires learning a multi-step decision policy, making RL a natural training framework.

SFT can imitate static demonstrations, but it is costly and does not directly teach exploratory search behavior; RL instead lets an LLM improve a policy through trial and error (Chu et al., 2025). Value-free methods such as GRPO (Shao et al., 2024) are attractive in this regime because they reduce the memory and modeling overhead of value estimation. Yet interactive search exposes two specific weaknesses that motivate our contribution. First, GRPO applies token-level ratios to sequence-level rewards, a mismatch that produces high-variance gradients and unstable updates (Zheng et al., 2025). Second, sparse binary search rewards frequently create rollout groups that are entirely correct or entirely incorrect, causing the group-relative advantage to vanish or become unusable and wasting samples (Yu et al., 2025; Liu et al., 2025). These two failure modes explain why RL search agents can collapse or plateau despite the promise of direct policy learning (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025).

We introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)** to make these two fixes explicit and jointly operational. The first component, drawn from the sequence-level perspective of GSPO (Zheng et al., 2025), aligns the optimization unit with the trajectory-level reward and thereby targets the instability of long-horizon reasoning. The second component, inspired by DAPO (Yu et al., 2025), dynamically retains only rollout groups containing both successful and unsuccessful trajectories, ensuring that $\hat{A}_i$ supplies a usable learning signal. The contribution is the integration of these ideas into a single RL procedure for multi-turn search and reasoning. Empirically, this structure matters: our 7B agent achieves a **34.1% relative improvement** over a leading 7B baseline (Jin et al., 2025b) and, on HotpotQA, surpasses the corresponding 14B system (Jin et al., 2025a) by **nearly 9% relative** (0.613 vs. 0.563).

For reviewers, the paper’s contributions are organized as follows:

- **Algorithmic contribution:** We propose DSPO, an RL algorithm for agentic search that unifies **sequence-level optimization** for stable policy updates with **dynamic outcome-based filtering** for informative sparse-reward training batches.

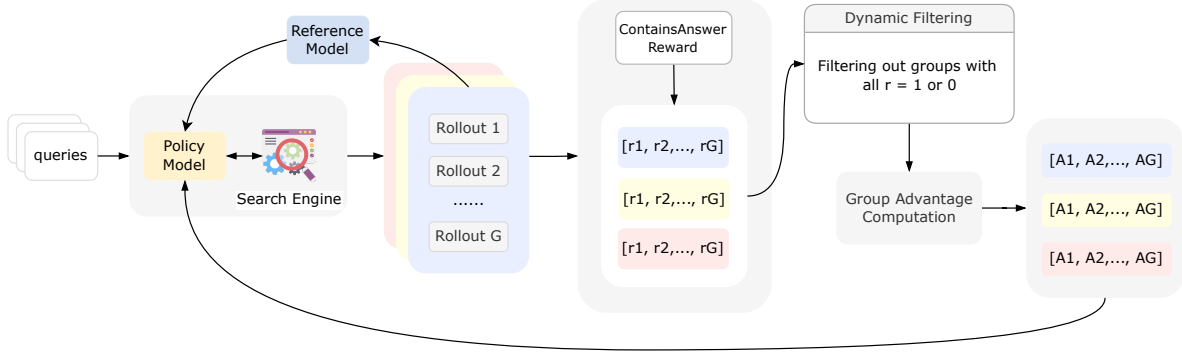


Figure 1: Contribution-level overview of the DSPO training loop. For each query, the policy generates G search-and-reasoning trajectories and receives sparse terminal rewards. The **dynamic filter** removes homogeneous-outcome groups and continues sampling until the batch contains informative groups, so each update has a non-zero advantage signal. The policy is then updated with the sequence-level objective, aligning optimization with trajectory-level reward.

- **Empirical contribution:** We show that DSPO yields substantial QA gains: the 7B model improves over a comparable 7B baseline by **34.1%** and outperforms the prior 14B counterpart on HotpotQA by nearly **9% relative** (0.613 vs. 0.563).
- **Analysis contribution:** We isolate why the gains occur through ablations and training-dynamics analysis, showing that DSPO stabilizes learning and that the reported improvements arise even with a basic BM25 retriever.

2 RELATED WORK

2.1 RL FOR LLMs

Prior RL-for-LLM work supplies the ingredients that motivate DSPO, but not the complete recipe for stable agentic search. RLHF methods typically rely on PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017), while DPO simplifies alignment through direct optimization (Rafailov et al., 2023). GRPO (Shao et al., 2024) further reduces overhead by replacing value estimation with group-relative rewards, but its token-level objective can be unstable (Liu et al., 2025; Cui et al., 2025). Recent variants address pieces of this problem: GSPO moves the objective to the sequence level to match the reward unit (Zheng et al., 2025), DAPO improves sparse-reward learning through dynamic outcome-based sampling (Yu et al., 2025), and GMPO uses a geometric-mean aggregation to reduce sensitivity to token-level outliers (Zhao et al., 2025). DSPO’s contribution is to combine the sequence-level and dynamic-filtering principles in the setting where both are needed simultaneously: autonomous search agents whose trajectories are long, sparse-rewarded, and prone to collapse or bottlenecks in our experiments.

2.2 LLMs WITH AGENTIC RETRIEVAL

RAG addresses static LLM knowledge by retrieving external information at inference time (Lewis et al., 2020; Gao et al., 2023); classic systems retrieve documents, often with dense retrievers, and concatenate them into the generation context (Karpukhin et al., 2020). This pipeline is useful but fixed, leaving limited room for autonomous multi-turn decisions. Agentic retrieval instead lets LLMs plan, search, and reason iteratively: ReAct couples reasoning with tool use (Yao et al., 2023), AutoGen supports collaborative multi-agent workflows (Wu et al., 2024), and Agentic Reasoning integrates external tools for streamlined reasoning (Wu et al., 2025). RL-based search agents (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) are especially relevant because they train the interleaving of reasoning and search directly, without requiring a hand-engineered multi-agent pipeline. DSPO contributes at this training layer: it preserves the end-to-end RL paradigm while addressing the instability and performance limitations that arise in complex interactive search environments.

3 METHODOLOGY

This section makes the algorithmic contribution precise. We first cast agentic search as an RL problem and identify where standard policy-gradient objectives misalign with sparse, trajectory-level search rewards. We then introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, separating its two contribution-bearing components—sequence-level optimization for stability and dynamic filtering for sample efficiency—before presenting the integrated training objective and algorithm.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. We begin from the standard policy-gradient baseline to clarify what DSPO changes. PPO (Schulman et al., 2017), widely used for LLM alignment, updates a policy π_θ with samples from an old policy $\pi_{\theta_{\text{old}}}$ and a clipped surrogate objective. In its token-averaged form, the objective is:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

Here, $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. The limitation, from the perspective of agentic search, is that PPO requires a separately trained value model to estimate token-level advantages $\hat{A}_t$ with Generalized Advantage Estimation (GAE) (Schulman et al., 2015), adding memory cost and another potential source of instability.

GRPO (Shao et al., 2024) removes this value-model requirement by sampling a group of G responses $\{y_i\}_{i=1}^G$ for each prompt x and normalizing each response reward against group statistics. This makes GRPO a strong starting point for efficient RL, but it retains the token-level optimization structure that DSPO later revises:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

This formulation reveals the mismatch DSPO targets: every token in y_i receives the same response-level advantage $\hat{A}_i$, yet the clipping and ratios remain token-level.

Agentic Search as a Markov Decision Process. We next define the search task in a way that makes the contribution requirement visible: the agent receives only trajectory-level feedback while making many intermediate search and reasoning decisions. We formalize this process as a discrete-time, finite-horizon Markov Decision Process (MDP).

- **State (s_t):** At turn t , the state s_t encodes the full interaction history: the initial question q , prior thoughts and search queries, and retrieved evidence returned by the environment. This growing state is central to our method framing because later decisions depend on how earlier evidence is interpreted, creating long-horizon dependencies that motivate sequence-level optimization.
- **Action (a_t):** An action is a full textual segment generated by π_θ : free-form reasoning followed by either a search decision or a final answer. The segment ends with a `</tool_call>` token to trigger search results inside `</tool_response>`, or with a `</answer>` token to terminate the trajectory. This action design makes the learned policy contribution explicit: the agent must learn both *what* to generate and *when* to search, producing variable-length trajectories.
- **Policy (π_θ):** The policy π_θ is the underlying LLM, which generates tokens autoregressively conditioned on the full state. Following Jin et al. (2025b), optimization is applied only to model-generated thoughts and actions, with retrieved environmental content masked out. This keeps the learning target aligned with DSPO’s contribution: improve the agent’s decisions over long sequences rather than train it to reproduce search results.

- **Trajectory** (τ): A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records the complete reasoning and tool-use path for a question, including model actions and environment-returned search results. Since reward is assigned only after the full trajectory, this definition directly motivates DSPO’s sequence-level update: early search choices and later answer generation are evaluated together.
- **Reward** ($R(\tau)$): We use the sparse terminal reward that creates the sample-efficiency challenge DSPO addresses: a trajectory receives $R = 1$ if the final answer contains the ground-truth text, and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we treat the search engine $\mathcal{S}$ as part of the environment rather than as a fixed preprocessing module. The policy LLM π_θ must therefore learn complete trajectories that interleave reasoning with calls to $\mathcal{S}$. The training goal is to maximize expected task reward while using a KL penalty to keep the learned policy close to a reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

The notation $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ emphasizes that y is not a single-pass completion: it is a multi-step trajectory shaped jointly by model-generated tokens and search results returned by $\mathcal{S}$.

Motivation for DSPO. Search-R1 (Jin et al., 2025b) shows that agentic search can be framed as RL, but this framing also exposes why conventional PPO- or GRPO-style updates are insufficient. The first failure mode is an optimization-unit mismatch: rewards are assigned to whole trajectories, whereas token-level ratios amplify high-variance updates across long search paths (Zheng et al., 2025). The second failure mode is signal sparsity: binary terminal rewards often produce all-success or all-failure groups, yielding abnormal or zero advantages and reducing sample efficiency (Yu et al., 2025; Liu et al., 2025). DSPO is designed around these two reviewer-visible claims: stabilize the update at the sequence level, and ensure that each retained batch contains a useful learning signal.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO’s contribution is the joint use of two mechanisms that map directly onto the two failure modes above. First, it performs policy optimization at the sequence level, aligning the update with trajectory-based rewards. Second, it dynamically filters rollout groups so each retained training batch contains a high-quality, non-zero learning signal. Figure 1 shows how these components interact in the full training loop.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

The first DSPO mechanism targets training stability. Inspired by GSPO (Zheng et al., 2025), we replace token-level importance ratios with a sequence-level ratio so that clipping and reward assignment operate at the same granularity. For response y_i , the sequence-level importance ratio $s_i(\theta)$ is the geometric mean of token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

The length normalization keeps $s_i(\theta)$ in a comparable numerical range across short and long trajectories, which reduces variance and makes clipping meaningful for multi-turn search.

Gradient Analysis. The gradient comparison isolates the stability contribution. In the unclipped token-level GRPO objective, each token log-probability gradient is scaled by its own noisy weight $r_{i,t}(\theta)$. DSPO instead scales the sequence’s log-probability gradient with one trajectory-level weight $s_i(\theta)$:

$$\nabla_\theta J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_\theta J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (8)$$

This single correction factor prevents token-level ratio noise from accumulating across the trajectory, which is the mechanism behind DSPO’s stability claim. The complementary dynamic filter then ensures that the shared advantage $\hat{A}_i$ is meaningful by retaining rollout groups with both successes and failures.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The second DSPO component addresses sample efficiency rather than gradient variance. Because the reward is sparse and binary, group-based advantage estimation fails whenever all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$): the normalized advantage $\hat{A}_i$ becomes zero or undefined, and the batch contributes little or no useful gradient signal.

DSPO therefore incorporates a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, we retain only trajectory groups that contain both successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This criterion guarantees non-zero reward variance inside every retained training group. As a result, dynamic filtering converts sparse terminal feedback into consistently informative group-relative advantages for each policy update.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

The final objective combines the two contribution-bearing components: advantages are computed only on dynamically retained groups, and policy updates use the sequence-level ratio. For each valid group from $\mathcal{D}_{\text{filtered}}$, we compute $\hat{A}_i$ with group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_θ is then updated by maximizing the DSPO objective:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

Here, s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. During likelihood calculation, we follow Jin et al. (2025b) and mask all tokens retrieved from the search tool, so the optimization targets the model’s reasoning and tool-use decisions rather than copying environmental text. Algorithm 1 summarizes the resulting DSPO training procedure.

4 EXPERIMENTS

The experiments are organized to test the paper’s three empirical contributions. First, we evaluate whether DSPO improves answer accuracy on challenging QA benchmarks. Second, we test whether its training dynamics avoid the collapse observed in token-level alternatives. Third, we use ablations to determine whether the full gains require both contribution-bearing components: sequence-level optimization and dynamic outcome filtering.

4.1 EXPERIMENTAL SETUP

To keep the evaluation focused on the optimizer rather than prompt engineering, we follow Search-R1 (Jin et al., 2025b). Table 1 shows the fixed prompt template used to specify the search-agent interface, including `<think>`, `<tool_call>`, and `<answer>` actions.

Benchmarks and Baselines. Our benchmark design follows the established Search-R1 protocol (Jin et al., 2025b) so that performance differences can be attributed to DSPO rather than to a new evaluation setup. We train on the combined training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018), then evaluate generalization on seven QA test sets: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

The comparison suite is designed to separate external competitiveness from internal mechanism validation. For external baselines, we use Qwen2.5-7B and 14B Search-R1 models trained with PPO and GRPO (Jin et al., 2025b;a).

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```

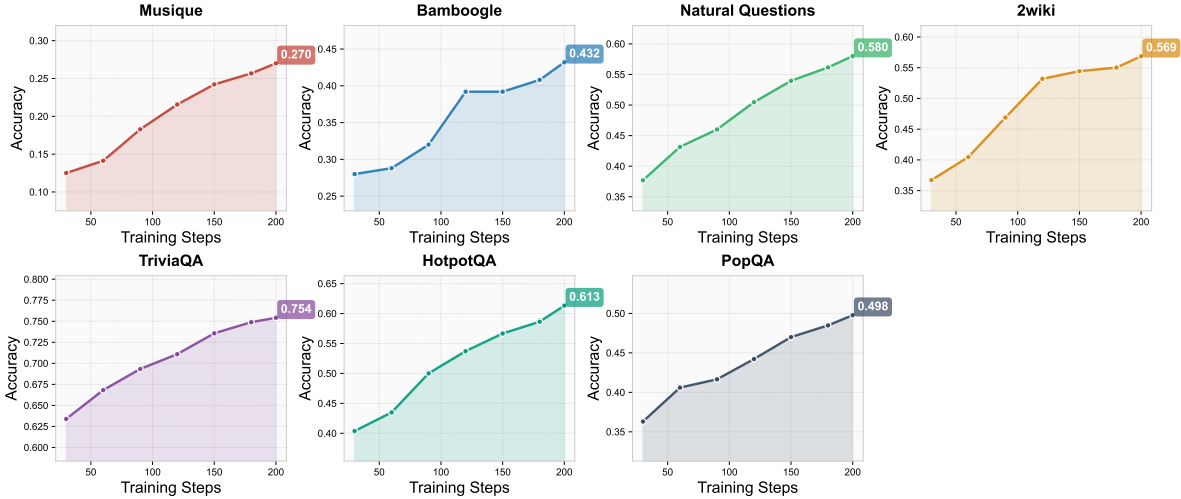


Figure 2: **Validation performance contribution of DSPO across seven benchmarks during training.** Accuracy rises steadily with training, showing that DSPO’s stable reward gains transfer to benchmark generalization rather than remaining confined to the training reward.

For ablations, we remove each DSPO component in turn: **DSPO w/o dynamic filter** corresponds to the GSPO-style sequence-level objective without filtering (Zheng et al., 2025), and **DSPO w/o sequence-level opt.** reverts to a strong token-level DAPO-style policy (Yu et al., 2025).

For implementation, all 7B training starts from Qwen2.5-7B-Instruct. We build on the VeRL framework (Sheng et al., 2025) and adapt its Search-R1-like example code and scripts to implement DSPO. Because our experiments use a modified reward function, we retrain the PPO and GRPO Search-R1 baselines under the same conditions; these retrained models serve as the primary external comparison points.

Implementation and Evaluation. To isolate DSPO’s contribution from retrieval improvements, every RL experiment uses the same standard BM25 retriever. All methods are trained with a sparse binary reward based on substring Exact Match (subEM) of the final answer, and subEM is also the primary evaluation metric. This controlled setup makes the reported gains evidence about the learned policy and optimizer rather than about a stronger search backend.

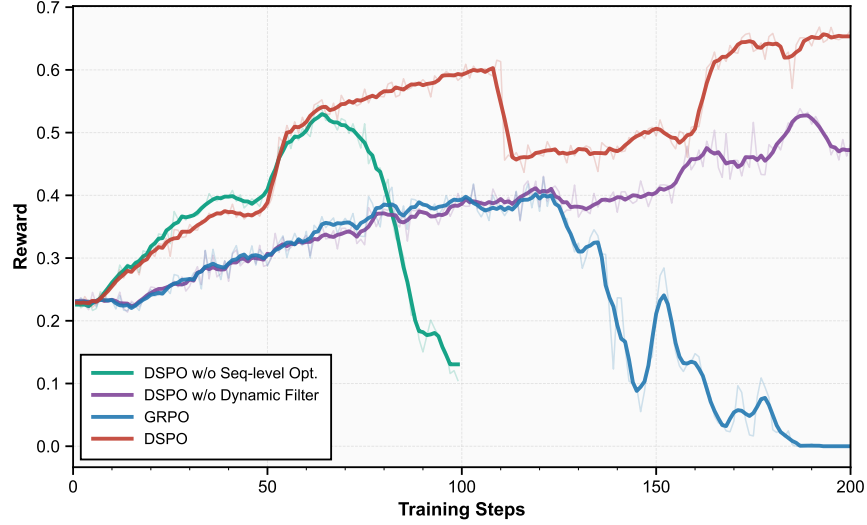


Figure 3: **Training-dynamics evidence for DSPO’s two components.** DSPO (red) improves stably and monotonically; token-level variants (green, blue) collapse, isolating the value of sequence-level optimization, while the sequence-level variant without dynamic filtering (purple) plateaus at a lower reward, isolating the value of the filter.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: Fixed prompt template used to expose the reasoning, search-call, and answer actions in all experiments.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 is structured to foreground both claims at once: DSPO’s competitiveness against Search-R1 baselines and the necessity of its two components. Presenting main results and ablations together makes the contribution easy to audit: reviewers can compare the full method to external systems from Search-R1 (Jin et al., 2025b;a), then immediately inspect what is lost when dynamic filtering or sequence-level optimization is removed. Table 2 therefore functions as both the main comparison and the mechanism check.

Comparison with Baselines. The headline result in Table 2 is that DSPO improves performance without relying on larger scale. The DSPO-trained 7B agent reaches an average score of 0.531, a 34.1% relative improvement over the same-sized Search-R1 GRPO baseline. It also slightly exceeds the average scores of the larger Search-R1 14B models trained with either GRPO or PPO. This comparison supports the paper’s central empirical contribution: DSPO’s optimization structure, not merely parameter count, drives the observed gains.

Analysis of Ablations. The ablations make the mechanism-level contribution explicit. Removing the dynamic filter (‘w/o dynamic filter’) reduces the average score to 0.313, showing that a sequence-level objective alone is not enough when homogeneous-reward batches starve the update of useful advantages. Removing sequence-level optimization (‘w/o sequence-level opt.’) yields an average score of 0.406; this token-level variant is stronger than the filter-less variant but remains far below full DSPO and, as the training-dynamics analysis shows, is vulnerable to collapse. Together, these results support the intended division of labor: sequence-level updates provide stability, while dynamic filtering makes sparse rewards learnable.

Beyond aggregate metrics, DSPO-trained agents exhibit the qualitative behaviors expected from a learned search policy: recognizing irrelevant results, reformulating queries, and verifying information across turns. Appendix A provides representative trajectories, and these behaviors emerge from pure RL training with DSPO.

Table 2: Main empirical contribution and component ablation on seven QA benchmarks. Search-R1 baselines include 7B and 14B models trained with GRPO and PPO (Jin et al., 2025b;a); DSPO ablations remove either dynamic filtering or sequence-level optimization. Original Search-R1 EM scores are shown in parentheses, while retrained scores under our adjusted reward provide the controlled comparison. Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	0.754
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

4.3 ANALYSIS OF TRAINING DYNAMICS

We next test the stability contribution directly by comparing reward trajectories for DSPO, its ablations, and key baselines. Figure 3 turns the algorithmic claims into observable training behavior: stable improvement should appear only when both sequence-level optimization and dynamic filtering are present.

DSPO (red) shows the desired pattern: a smooth, monotonic rise that converges to the highest reward level. The token-level methods—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—instead collapse after a brief improvement, matching the predicted instability from high-variance token-level updates. DSPO w/o Dynamic Filter (purple) remains more stable because it keeps the sequence-level objective, but it plateaus at a lower reward because it lacks consistently informative batches. These curves visually separate the two contributions: sequence-level optimization prevents collapse, and dynamic filtering raises the attainable learning signal.

We also check that the reward improvement corresponds to benchmark generalization rather than reward hacking. Figure 2 shows that DSPO’s validation accuracy on NQ, HotpotQA, and the remaining benchmarks rises consistently alongside the reward curve, supporting the claim that DSPO learns a generalizable search-and-reasoning policy.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

Finally, we ask whether the contribution is limited to the main 7B search setting or transfers across scale and task family. We therefore evaluate larger search models and single-turn mathematical reasoning.

Scalability to Larger Models. To test scalability, we train Qwen2.5-14B-Instruct with DSPO. Table 3 shows that the 14B DSPO model reaches 60.6% average accuracy, improving over the GRPO-14B baseline at 53.0% by a relative **14.3%**. This result indicates that DSPO’s stability and filtering benefits continue to matter at larger parameter scale and can leverage additional model capacity.

Generalization to Mathematical Reasoning. We also probe domain generalization by applying DSPO to single-turn mathematical reasoning with Qwen2.5 and Qwen3 models. Table 4 compares Math500 and Olympiad-Bench results, where DSPO exceeds GRPO for both the 7B and 4B settings. These results suggest that DSPO’s optimization principles are not restricted to agentic search and can benefit broader reasoning domains.

5 CONCLUSION

This paper contributes a targeted RL optimizer for autonomous LLM search agents, with the contribution structured around the two main obstacles in this setting: unstable token-level updates and inefficient sparse-reward learning. Dynamic-filter Sequence-level Policy Optimization (DSPO) addresses these obstacles by pairing sequence-level policy optimization, which aligns updates with trajectory-level rewards, with dynamic outcome-based filtering, which

Table 3: Scalability contribution on Qwen2.5-14B-Instruct, showing that DSPO’s gains persist at larger model scale. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Table 4: Generalization contribution on mathematical reasoning benchmarks, where DSPO outperforms GRPO beyond the search-agent setting. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

keeps training batches informative. Experiments across challenging QA benchmarks show that this structure improves accuracy and yields more stable training than prior methods.

Because DSPO learns from environmental feedback alone, it offers a practical route to capable search agents without supervised demonstrations. The same contribution framing also suggests clear next steps: stronger retrievers could be integrated on top of the optimizer, and the method could be extended to more complex multi-tool settings. More broadly, sparse rewards and unstable policy gradients also arise outside search; the mathematical-reasoning results provide initial evidence that DSPO’s two principles—matching the optimization unit to the reward signal and preserving signal density—may transfer to domains such as mathematics and code generation.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.

- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiakowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

Table A.1 complements the quantitative contribution by showing what the learned policy does during search. The representative trajectories illustrate three behaviors that emerge under DSPO training: query reformulation after irrelevant retrieval, follow-up verification, and direct extraction of answer evidence from retrieved results.

Table A.1: Qualitative evidence for DSPO’s learned search-and-reasoning policy. The trajectories highlight (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let’s try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to <u>Cornell University</u>, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician